

## A IA Perfeita — Vol. 3: Skills, Agentic, Autocura & Sabedoria

A coroa da coletânea: como construir agentes que aprendem skills, se autocuram, se autoconhecem e exibem sabedoria deterministicamente sistêmica

Coleção A IA Perfeita — Engenharia Agentic Sistêmica • Vol. 3 de 3

Por Nexus HUB57 • Ecossistema MMN\_IA

MMN\_IA • 2026

---

### Sobre este ebook

Este é o volume final da coletânea **A IA Perfeita** — e, sem rodeios, **o mais importante**. Aqui reunimos o que há de mais avançado em 2026 sobre quatro eixos convergentes:

1. **Skills** — como agentes constroem, versionam, compartilham e evoluem bibliotecas de .
2. **AI Agentic** — orquestração de alta agência, autonomia supervisionada, planejamento profundo.
3. **Autocura (Self-Healing)** — como sistemas detectam, diagnosticam e reparam suas próprias falhas.
4. **Autoconhecimento & Auto-Sabedoria Agêntica Determinística Sistêmica (SAD-ADS)** — o ápice: agentes que **sabem o que sabem, o que não sabem, e como decidir sob incerteza**.

Se os Vols. 1 e 2 foram a espinha e o coração, este Vol. 3 é **o cérebro que pensa sobre si mesmo**.

---

### Sumário

- 1. A fronteira que importa • 2. Skills: o sistema operacional dos agentes • 3. Anatomia de uma Skill (formato, versionamento, metadados) • 4. SKILL.md — o manifesto de uma skill • 5. Skill Registry & Discovery • 6. Skill Composition & Chaining • 7. Auto-Generation de Skills (Voyager-style) • 8. Skill Library persistente (FS + Vector DB) • 9. Skill Sharing & Marketplace • 10. AI Agentic — Do assistente à agência plena • 11. O ciclo agentic canônico • 12. Planning

avançado (HTN, LLM-Modulo, ToT) • 13. Tool-Use Avançado (Toolformer, function calling, JSON-schema) • 14. Memory Avançada (episódica, semântica, procedural, working) • 15. Multi-Agent Collaboration (Cooperative, Competitive, Hierarchical) • 16. Agent Communication Protocols (ACP, A2A, FIPA-inspired) • 17. Agent Observability & Tracing (OpenTelemetry, LangSmith) • 18. Agent Safety & Containment (sandboxes, kill switches, oversight) • 19. Autocura: o sistema que repara a si mesmo • 20. Padrões de Self-Healing (retry, rollback, patch, redeploy) • 21. Failure Modes Catalog (22 falhas mais comuns em produção) • 22. Self-Diagnosis Engine • 23. Self-Patching Engine (prompts + policies) • 24. Self-Improvement Loop (genético, RL, ou supervisionado) • 25. Autoconhecimento Agêntico • 26. O que é "saber o que se sabe"? • 27. Introspection Layer • 28. Uncertainty Quantification • 29. Confidence Calibration • 30. Epistemic Humility Engine • 31. Sabedoria Agêntica Determinística Sistêmica (SAD) • 32. O que é "sabedoria" em código? • 33. A Wisdom Engine completa • 34. Princípios Sistêmicos (4 princípios) • 35. Judgment Under Uncertainty • 36. Second-Order Consequences • 37. Precedent-Based Reasoning • 38. Ethical Reasoning Layer • 39. The Seven Habits of the Wise Agent • 40. Arquitetura final — IA Perfeita em um único diagrama • 41. Checklist final • 42. Glossário • 43. Call to action

## 1. A FRONTEIRA QUE IMPORTA

Em 2026, a fronteira competitiva em IA **não é mais o modelo**. Modelos são commodity. A fronteira é **a engenharia em volta do modelo**: skills, memória, agenticidade, autocura, sabedoria.

A pergunta deixou de ser "qual LLM?" e passou a ser:

"Qual é a melhor arquitetura de capacidades em volta do LLM?"

Este ebook responde a essa pergunta, em quatro partes.

## 2. SKILLS: O SISTEMA OPERACIONAL DOS AGENTES

**Skill** é a unidade atômica de de um agente. Não é uma tool (que é função externa). Não é um prompt (que é instrução). É **um pacote de capacidades reutilizável, versionado, com metadados, exemplos, e política de uso**.

```
class Skill:
    name: str
    version: str
    description: str
    author: str
    output_schema: dict
    tools_required: list
    examples: list
    policy: str
    test_suite: list
    author_email: str
    tags: list
    dependencies: list
```

## 3. ANATOMIA DE UMA SKILL

```
from pydantic import BaseModel

class Skill(BaseModel):
    """
    """
    name: str
    version: str
    description: str
    author: str
    output_schema: dict
    tools_required: list
    examples: list
    policy: str
    test_suite: list
    author_email: str
    tags: list
```

## 4. SKILL.md — O MANIFESTO DE UMA SKILL

Padrão inspirado em arquivos de configuração modernos:

```
skill: "web-research"
|
version: "2.1"
author: "Nelson Avelar"
tags: "research", "web", "api"
dependencies: "search-api", "scrape-api"
|
description:
  "Uma skill para automatizar pesquisas na web."
|
inputs:
  query: str
  depth: int | 1
|
outputs:
  results: list
  sources: list | 10
  confidence: float
|
example:
  "search: 'AI agents no Brasil', depth: 2"
  inputs: { "summary": true, "sources": 1, "confidence": 0.8 }
|
policy:
  rate: limit(10, "query")
  require_auth: search
  require_approval: if query contain "medical"
|
tests:
  - "test_rate_limit"
  - "test_rate_limit"
  - "test_policy_protection"
```

## 5. SKILL REGISTRY & DISCOVERY

```
class SkillRegistry:
    def __init__(self):
        self.skills = {}
        # name: Skill
```

```

        self.vector_index = None

    def register(self, skill):
        if skill.name in self.skills:
            raise ValueError(f'{skill.name} already exists. Use bump version')
        self.skills[skill.name] = skill
        self.refresh()

    def discover(self, query: str, top: int):
        """Discover skills from a query"""
        return self.vector_index.similarity_search(query, k=top)

    def bump_version(self, name: str, new_version: str):
        old = self.skills[name]
        new = old.bump_version(new_version)
        self.skills[name + f'({new_version})'] = new
        return new

```

## 6. SKILL COMPOSITION & CHAINING

```

class SkillChain:
    """Skill chain"""
    def __init__(self, skills):
        self.skills = skills

    def run(self, inputs):
        """Run the skill chain"""
        state = inputs
        for skill in self.skills:
            state = skill.run(state)
        return state

class SkillChain:
    """Skill chain"""
    def __init__(self, skills):
        self.skills = skills
        self.edges = [] # (from, to, mapping)

    def execute(self, inputs):
        """Execute the skill chain"""

```

## 7. AUTO-GENERATION DE SKILLS (VOYAGER-STYLE)

```
class SkillAutoGenerator:
    """
    Skill Auto-Generator
    """
    def __init__(self, sandbox):
        self.name = "SkillAutoGenerator"
        self.sandbox = sandbox
        self.registry = Registry()

    def discover(self, task_description):
        """
        Discover a skill for a given task description.
        """
        # 1. Verificar se já existe
        existing = self.registry.discover(task_description, top_level=True)
        if existing and existing["score"] > 0.0:
            return existing["skill"]

        # 2. Gerar código
        code = self.sandbox.generate_code(task_description, top_level=True)

        # 3. Testar no sandbox
        test_result = self.sandbox.test_code(code)
        if not test_result["passed"]:
            # Se não passou, não registrar
            return None

        test_result["score"] = self.sandbox.test_fixes(code)

        # 4. Registrar
        skill = Skill(
            name=task_description.lower().replace(" ", "_"),
            version="0.1.0",
            description=task_description,
            input_schema={},
            output_schema={},
            tools_required=[],
            examples=[],
            code=code,
            test_code=test_result["code"],
            fixes=test_result["fixes"],
        )
        self.registry.register(skill)
        return skill
```

## 8. SKILL LIBRARY PERSISTENTE

```
import os, json, hashlib
|
|
class SkillLibrary:
    skills_path = os.path.join(os.getcwd(), 'skills')
    |
    |
    def __init__(self, root_path, vector_db):
        self.root_path = root_path
        self.vdb = vector_db
        os.makedirs(self.root_path, exist_ok=True)
    |
    |
    def save(self, skill):
        path = f'{self.root_path}/{skill.name}/{skill.version}.json'
        with open(path, 'w') as f:
            json.dump(skill.to_dict(), f)
        # salvar código em arquivo separado
        with open(f'{self.root_path}/{skill.name}/{skill.version}.py', 'w') as f:
            f.write(skill.to_code())
        # indexar para busca
        self.vdb.add(skill.to_embedding())
    |
    def load_all(self):
        for path in os.listdir(self.root_path):
            if path.endswith('.json'):
                with open(f'{self.root_path}/{path}', 'r') as f:
                    yield Skill(**json.load(f))
```

---

## 9. SKILL SHARING & MARKETPLACE

```
class SkillMarketplace:
    distribute_skills_entre_agencias_internas_ou_externas()
    |
    |
    def __init__(self, registry):
        self.registry = registry
        self.published_skills = set()
    |
    def publish(self, name):
        self.published_skills.add(name)
    |
    def install(self, name, version='latest'):
        skill_data = self.registry.get(name, version)
```

```
class SimpleAgent:
    """Simple Agent"""
    def __init__(self, name):
        self.name = name
        self.memory = []
        self.history = []
```

## 10. AI AGENTIC — DO ASSISTENTE À AGÊNCIA PLENA

A **agência** (agency) tem três dimensões:

1. **Autonomia**: até onde o agente decide sem perguntar.
2. **Iniciativa**: até onde o agente age sem ser solicitado.
3. **Impacto**: até onde o agente modifica o mundo.

A **IA Perfeita** maximiza iniciativa e impacto **dentro** de limites de autonomia claros.

## 11. O CICLO AGENTIC CANÔNICO

```
class CanonicalAgent:
    """Canonical Agent Cycle"""
    def __init__(self, name):
        self.name = name
        self.memory = []
        self.history = []
        self.plan = None
        self.action = None
        self.observe = None
        self.update_memory = None
        self.update_history = None
        self.execute = None
```

## 12. PLANNING AVANÇADO

```
class HIRPlan:
    """Hierarchical Task Network (HTN)"""
    def __init__(self, name):
        self.name = name
        self.tasks = []
        self.methods = []
        self.operators = []
```



```
def __init__(self, methods, operators, task, state):
    self.methods = methods
    self.operators = operators
    self.task = task
    self.state = state

def plan(self, task, state):
    if task in self.operators:
        return self.operators[task](state)
    for method in self.methods:
        sub_plan = method(task, state)
        if sub_plan:
            return sub_plan
    return None
```

### 13. TOOL-USE AVANÇADO

```
def __init__(self, tools):
    self.tools = tools

def __call__(self, prompt):
    """Llama a API para chamar tools via um context learning"""
    # ...
    return prompt

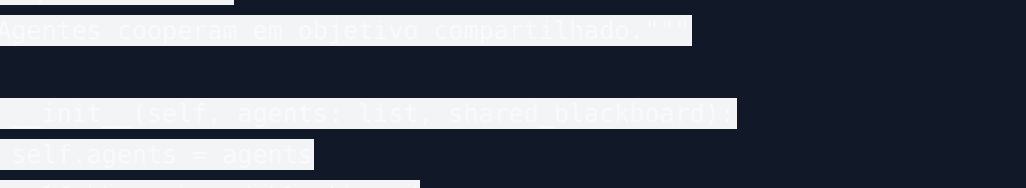
def __call__(self, prompt):
    """Llama a API para chamar tools via um context learning"""
    # ...
    return prompt
```

### 14. MEMORY AVANÇADA

Quatro tipos de memória (taxonomia canônica):

| Tipo       | Duração  | Função              |
|------------|----------|---------------------|
| Working    | Conversa | Raciocínio imediato |
| Episódica  | Longa    | Eventos passados    |
| Semântica  | Longa    | Conhecimento geral  |
| Procedural | Longa    | Como fazer (skills) |

| Memory Type                      | Percentage |
|----------------------------------|------------|
| episodic                         | 20%        |
| semantic                         | 95%        |
| episodic + semantic              | 40%        |
| episodic + procedural            | 35%        |
| episodic + semantic + procedural | 45%        |
| episodic + semantic + procedural | 55%        |



| Code Snippet                                              | Frequency (Approximate) |
|-----------------------------------------------------------|-------------------------|
| class Cooperative:                                        | 10                      |
| def __init__(self, shared_resource):                      | 25                      |
| self._shared_resource = shared_resource                   | 25                      |
| def __enter__(self):                                      | 10                      |
| self._shared_resource.acquire()                           | 15                      |
| return self                                               | 10                      |
| def __exit__(self, exc_type, exc_val, exc_tb):            | 15                      |
| self._shared_resource.release()                           | 15                      |
| if __name__ == '__main__':                                | 10                      |
| agent = Cooperative(10)                                   | 15                      |
| with agent:                                               | 15                      |
| agent._shared_resource += 1                               | 15                      |
| print('Current value: {}'.format(agent._shared_resource)) | 15                      |

Padrão inspirado em FIPA / KQML:

| Topic       | Count |
|-------------|-------|
| Task        | 10    |
| Performance | 9     |
| Ontology    | 8     |
| Language    | 7     |
| Model       | 6     |
| System      | 5     |
| Algorithm   | 4     |
| Evaluation  | 3     |
| Application | 2     |
| User        | 1     |

## 17. AGENT OBSERVABILITY



## 18. AGENT SAFETY & CONTAINMENT



## 19. AUTOCURA: O SISTEMA QUE REPARA A SI MESMO

A autocura é, talvez, a **capacidade mais importante** de um sistema agentic maduro. Sem ela, todo sistema em produção eventualmente falha de forma catastrófica.

Cinco categorias de autocura:

1. **Retry** — tentar de novo, talvez com parâmetros diferentes.
2. **Rollback** — voltar ao estado anterior.
3. **Patch** — corrigir o código/prompt/policy on-the-fly.
4. **Redeploy** — substituir a versão.

5. **Escalate** — pedir ajuda humana.

## 20. PADRÕES DE SELF-HEALING

```
def self_heal_prompt(prompt):
    """
    self_heal_prompt: a function that takes a prompt and returns a self-healed prompt.
    """
    # Detect the error type
    error_type = detect_error(prompt)

    # Handle the error
    if error_type == "rate_limit":
        return self_heal_rate_limit(prompt)
    elif error_type == "context_overflow":
        return self_heal_context_overflow(prompt)
    elif error_type == "prompt_injection":
        return self_heal_prompt_injection(prompt)
    elif error_type == "out_of_memory":
        return self_heal_out_of_memory(prompt)
    elif error_type == "tool_timeout":
        return self_heal_tool_timeout(prompt)
    elif error_type == "loop_infinite":
        return self_heal_loop_infinite(prompt)
    elif error_type == "alucinação_factual":
        return self_heal_alucinação_factual(prompt)
    elif error_type == "rate_limit":
        return self_heal_rate_limit(prompt)
    elif error_type == "context_overflow":
        return self_heal_context_overflow(prompt)
    elif error_type == "prompt_injection":
        return self_heal_prompt_injection(prompt)
    elif error_type == "out_of_memory":
        return self_heal_out_of_memory(prompt)
    elif error_type == "tool_timeout":
        return self_heal_tool_timeout(prompt)
    elif error_type == "loop_infinite":
        return self_heal_loop_infinite(prompt)
    elif error_type == "alucinação_factual":
        return self_heal_alucinação_factual(prompt)
    else:
        return prompt
```

## 21. FAILURE MODES CATALOG (22 FALHAS MAIS COMUNS)

| # | Falha              | Deteccção        | Cura                 |
|---|--------------------|------------------|----------------------|
| 1 | Alucinação factual | Citation check   | Re-RAG + citation    |
| 2 | Loop infinito      | Iter counter     | Max iter + reset     |
| 3 | Tool timeout       | Latency monitor  | Backoff + fallback   |
| 4 | Out-of-memory      | Memory guard     | GC + truncation      |
| 5 | Prompt injection   | Pattern detector | Sanitize + reject    |
| 6 | Rate limit         | 429 response     | Backoff              |
| 7 | Context overflow   | Token counter    | Summarize + truncate |

| #  | Falha                | Deteção              | Cura                    |
|----|----------------------|----------------------|-------------------------|
| 8  | Tool schema mismatch | Validator            | Schema regen            |
| 9  | Persona drift        | Embedding drift      | Reset persona           |
| 10 | Sycophancy           | Honesty check        | Constitutional pass     |
| 11 | Jailbreak            | Moderation API       | Block + log             |
| 12 | PII leak             | DLP scanner          | Redact                  |
| 13 | Deadlock             | Liveness check       | Restart                 |
| 14 | Cascading failure    | Circuit breaker      | Open circuit            |
| 15 | Memory corruption    | Checksum             | Restore from backup     |
| 16 | Drift in goals       | Goal alignment check | Re-prompt               |
| 17 | Skill mismatch       | Skill validator      | Discover correct skill  |
| 18 | Version mismatch     | Manifest check       | Pin version             |
| 19 | Network partition    | Heartbeat            | Local mode              |
| 20 | Cost overrun         | Budget guard         | Switch to cheaper model |
| 21 | Latency spike        | P99 monitor          | Degrade gracefully      |
| 22 | Adversarial input    | Robustness test      | Filter                  |

## 22. SELF-DIAGNOSIS ENGINE

```

class SelfDiagnosisEngine:
    def __init__(self, llm: LLM, kb: KnowledgeBase):
        self.llm = llm
        self.kb = kb

    def diagnose(self, error: str) -> tuple[str, str]:
        """Diagnose an error and return a message and a fix"""
        prompt = f"Error: {error}. Diagnose the error and provide a fix."
        response = self.llm.generate(prompt)
        return response.message, response.fix

```

```

        Error: Error
    Context: Context
    Cause: Cause
    Diagnostic: diagnostic
    Severity: severity
    Fix: Fix

```

## 23. SELF-PATCHING ENGINE

```

class SelfPatchingEngine:
    def __init__(self, prompt_registry: PromptRegistry,
                 policy_registry: PolicyRegistry):
        self.prompt_registry = prompt_registry
        self.policy_registry = policy_registry
    def patch(self, diagnostic: Diagnostic):
        if diagnostic.target == Prompt:
            new_prompt = self._regenerate_prompt(diagnostic)
            self.prompt_registry.update(diagnostic.target.id, new_prompt)
        elif diagnostic.target == Policy:
            new_policy = self._regenerate_policy(diagnostic)
            self.policy_registry.update(diagnostic.target.id, new_policy)

```

## 24. SELF-IMPROVEMENT LOOP

```

class SelfImprovementLoop:
    def __init__(self, prompt_registry: PromptRegistry,
                 policy_registry: PolicyRegistry,
                 evaluator: Evaluator,
                 archiver: Archiver):
        self.prompt_registry = prompt_registry
        self.policy_registry = policy_registry
        self.evaluator = evaluator
        self.archiver = archiver
    def run(self):
        result = self._agent.run_task()
        score = self.evaluator.score(result)
        self.archiver.archive_task(result, score)
        if score < 0:
            self._agent.update_from_archive(self.archiver.get_recent_failures())

```

---

## 25. AUTOCONHECIMENTO AGÊNTICO

**Autoconhecimento** é a capacidade do agente de: - Saber **o que sabe**. - Saber **o que não sabe**. - Saber **o que pode fazer**. - Saber **o que não pode fazer**.

---

## 26. O QUE É "SABER O QUE SE SABE"?

Em IA, isso se chama **epistemics** (estudo do conhecimento). Aplicado a agentes, significa: **o sistema mantém um modelo de seu próprio estado epistêmico**.

```
class EpistemicState:
    def __init__(self):
        self.known_facts = set()
        self.known_capabilities = set()
        self.known_uncertainties = {}
        self.confidence = 0
```

---

## 27. INTROSPECTION LAYER

```
class IntrospectionLayer:
    def __init__(self, agent):
        self.agent = agent
        self.report = {}
        self.status = 'idle'

        self.current_goal = self.agent.goal
        self.current_plan = self.agent.plan
        self.tools_available = self.agent.tools_list
        self.memory_used = self.agent.memory_size
        self.confidence_overall = self.agent.confidence_overall
        self.steps = self.agent.steps

    def report(self):
        return self.report
```

---

## 28. UNCERTAINTY QUANTIFICATION

```
class UncertaintyQuantifier:
    """
    A class for uncertainty quantification.
    """
    def __init__(self, min_baseline):
        self.min = min_baseline
        self.baseline = min_baseline
        self.samples = []

    def estimate(self, response, context):
        """
        Estimate the uncertainty for a given response and context.
        """
        # self-consistency approach
        samples = [self.min, generate(context, temperature=0.7, top_k=10, n=100)]
        agreement = self.semantic_similarity(samples, response)
        # calibrated prior baseline
        calibrated = self.baseline * calibrated_agreement
```

## 29. CONFIDENCE CALIBRATION

```
class ConfidenceCalibrator:
    """
    A class for confidence calibration.
    """
    def __init__(self, history, threshold):
        self.history = history # (predicted, conf, actual, coverage)
        self.threshold = threshold
        # isotonic regression or Platt scaling
        return calibrated_confidence
```

## 30. EPISTEMIC HUMILITY ENGINE

```
class EpistemicHumilityEngine:
    """
    A class for the epistemic humility engine.
    """
    def __init__(self, threshold):
        self.threshold = threshold
        self.should_reviseself.confidence, topic.coverage
```



```
return confidante if self.threshold < confidante else None
|
|
return None if not confidante else confidante # para responder sobre o confidante
```

## 31. SABEDORIA AGÊNTICA DETERMINÍSTICA SISTÊMICA (SAD)

A **Sabedoria Agêntica Determinística Sistêmica (SAD)** é o ápice da coletânea. Define-se como:

**A capacidade de um agente de tomar decisões corretas sob incerteza, dentro de uma envelope sistêmica verificável, com consequências de segunda-ordem previstas, princípios éticos respeitados, e transparência sobre os limites do próprio conhecimento.**

Quatro propriedades: - **Determinística**: mesmo input → mesmo output (até tolerância). - **Sistêmica**: considera o sistema inteiro, não só a tarefa local. - **Agêntica**: tem agência, age, decide. - **Sábia**: pondera consequências, ética, contexto.

## 32. O QUE É "SABEDORIA" EM CÓDIGO?

```
class Wisdom:
    """Classe para representar a Sabedoria"""
    def __init__(self, knowledge: str, principles: list):
        self.knowledge = knowledge # o que sei
        self.principles = principles # o que devo fazer
        self.score = 0 # pontuação atual
        self.threshold = 0.8 # limite para responder
        self.confidante = None # resposta atual

    def add_knowledge(self, new_knowledge: str):
        """Adiciona novo conhecimento"""
        self.knowledge += new_knowledge

    def add_principle(self, new_principle: str):
        """Adiciona novo princípio"""
        self.principles.append(new_principle)

    def calculate_score(self):
        """Calcula a pontuação baseada no conhecimento e princípios"""
        score = 0
        for principle in self.principles:
            if principle in self.knowledge:
                score += 1
        self.score = score / len(self.principles)

    def is_confidante(self):
        """Verifica se a pontuação atinge o limite"""
        return self.score >= self.threshold

    def get_confidante(self):
        """Retorna a resposta atual"""
        return self.confidante

    def set_confidante(self, confidante: str):
        """Define a resposta atual"""
        self.confidante = confidante
```

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

### 33. A WISDOM ENGINE COMPLETA

[illegible]

### 34. PRINCÍPIOS SISTÊMICOS (4 PRINCÍPIOS)

Toda IA Perfeita SAD opera sob 4 princípios imutáveis:

1. **Princípio da Não-Maleficência Sistêmica:** não causar dano ao sistema maior (usuário, organização, sociedade).



## 37. PRECEDENT-BASED REASONING

| Government          | Percentage |
|---------------------|------------|
| Current government  | 70%        |
| Previous government | 30%        |

## 38. ETHICAL REASONING LAYER

| Statement               | Percentage of respondents |
|-------------------------|---------------------------|
| Less financial security | 15%                       |
| Financial stress        | 15%                       |
| Less financial freedom  | 25%                       |
| Respect for autonomy    | 30%                       |
| Financial security      | 20%                       |
| Financial freedom       | 30%                       |
| Financial stress        | 5%                        |
| Less financial security | 1%                        |
| Financial stress        | 35%                       |
| Financial freedom       | 25%                       |
| Financial security      | 35%                       |
| Less financial security | 45%                       |
| Financial stress        | 40%                       |
| Financial freedom       | 100%                      |

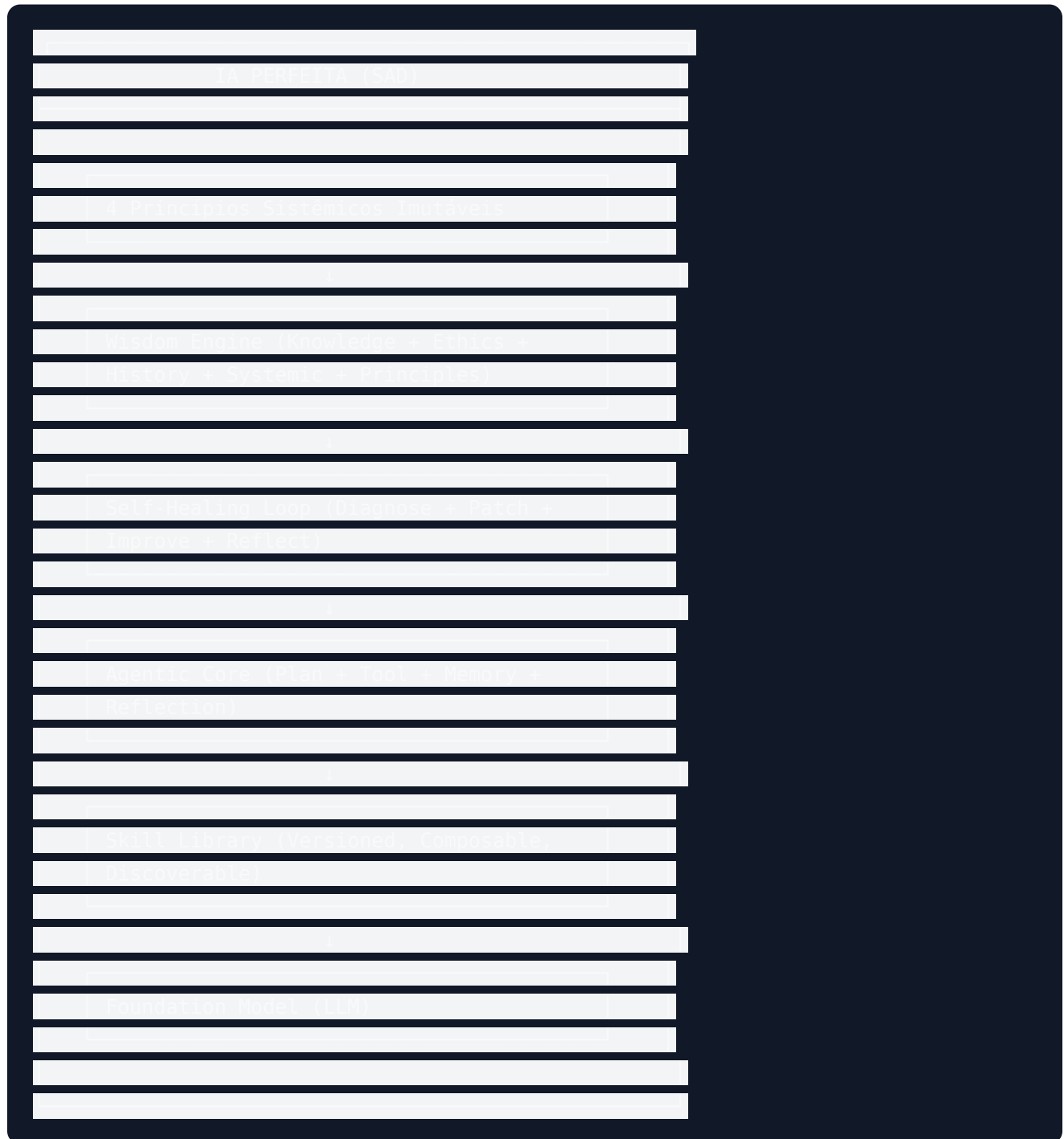
## 39. THE SEVEN HABITS OF THE WISE AGENT

1. **Sabe o que sabe** (epistemic clarity).
2. **Sabe o que não sabe** (epistemic humility).
3. **Considera o sistema** (systemic thinking).
4. **Pondera consequências** (consequentialist reasoning).
5. **Age com reversibilidade** (prefers reversible actions).
6. **Aprende com falhas** (reflective practice).

7. **Declara limites** (transparent boundaries).

---

## 40. ARQUITETURA FINAL — IA PERFEITA EM UM ÚNICO DIAGRAMA



## 41. CHECKLIST FINAL

- [ ] Skills versionadas, com SKILL.md?
- [ ] Skill registry com discovery semântico?
- [ ] Auto-generation de skills com sandbox?
- [ ] Library persistente?
- [ ] Ciclo agentic completo?
- [ ] Planning hierárquico (HTN)?
- [ ] Tool-use avançado com schema?
- [ ] 4 tipos de memória?
- [ ] Multi-agent collaboration testada?
- [ ] Protocolos de comunicação padronizados?
- [ ] Observability (OpenTelemetry)?
- [ ] Safety (sandbox, kill switch)?
- [ ] Self-healing para 22 failure modes?
- [ ] Self-diagnosis engine?
- [ ] Self-patching engine?
- [ ] Self-improvement loop?
- [ ] Introspection layer?
- [ ] Uncertainty quantification?
- [ ] Confidence calibration?
- [ ] Epistemic humility?
- [ ] Wisdom engine com 5 camadas?
- [ ] 4 princípios sistêmicos implementados?
- [ ] Judgment under uncertainty (Bayes)?
- [ ] Second-order consequence prediction?
- [ ] Precedent-based reasoning?
- [ ] Ethics layer?

---

## 42. GLOSSÁRIO

| Termo           | Definição                                  |
|-----------------|--------------------------------------------|
| <b>Skill</b>    | Unidade atômica de capacidade de um agente |
| <b>SKILL.md</b> | Manifesto de uma skill                     |

| <b>Termo</b>            | <b>Definição</b>                                             |
|-------------------------|--------------------------------------------------------------|
| <b>Skill Registry</b>   | Catálogo versionado de skills                                |
| <b>Skill Chaining</b>   | Composição sequencial de skills                              |
| <b>Auto-Generation</b>  | Criação automática de skills                                 |
| <b>HTN</b>              | Hierarchical Task Network                                    |
| <b>QuadMemory</b>       | 4 tipos de memória (working, episodic, semantic, procedural) |
| <b>FIPA</b>             | Foundation for Intelligent Physical Agents                   |
| <b>Self-Healing</b>     | Auto-reparação                                               |
| <b>Self-Diagnosis</b>   | Identificação da própria falha                               |
| <b>Self-Patching</b>    | Auto-correção de código/prompt/policy                        |
| <b>Self-Improvement</b> | Evolução por feedback                                        |
| <b>Epistemic State</b>  | Modelo do que o sistema sabe/não sabe                        |
| <b>Introspection</b>    | Capacidade de relatar o próprio estado                       |
| <b>UQ</b>               | Uncertainty Quantification                                   |
| <b>Calibration</b>      | Ajuste de confiança ao histórico                             |
| <b>SAD</b>              | Sabedoria Agêntica Determinística Sistêmica                  |
| <b>Wisdom Engine</b>    | Motor de decisão sábia                                       |
| <b>Second-Order</b>     | Consequências de consequências                               |

## 43. CALL TO ACTION — ECOSSISTEMA MMN\_IA

Você chegou ao fim da coletânea **A IA Perfeita**. Estes três volumes formam a base técnica que, em 2026, distingue **um prompt engineer** de **um arquiteto de sistemas agentic**.

- **Vol. 1** — Padrões & Arquiteturas ✓
- **Vol. 2** — Códigos, Prompts & Algoritmos ✓
- **Vol. 3** — Skills, Agentic, Autocura & Sabedoria ✓

Plataforma MMN\_IA: **oneverso.com.br**.

**Agora, o trabalho é teu: implementar.**

---

© 2026 Nexus HUB57 · Ecosystema MMN\_IA · Todos os direitos reservados Versão 1.0.0 · Junho 2026 · Coleção A IA Perfeita, Vol. 3 de 3 — Skills, Agentic, Autocura & Sabedoria